

G03 · Titans — 시험 때 기억하는 법을 배우다

이 권은 “test-time 에 스스로를 학습시키는 memory” 라는 이 계열의 원조, **Titans**(arXiv:2501.00663, Google Research, 2024) 를 다룬다. 핵심아이디어 하나만 가져가면 된다: memory 를 고정 크기 표 (table) 가 아니라 **읽는 도중에도 계속 학습시키는 작은 신경망**으로 만드는 것. 이권의 목표는 그 memory 가 무엇을 저장하고, 토큰마다 어떤 연산을 몇 번하며, 시스템 관점에서 KV cache 와 무엇이 달라지는지를 — 증명 없이, 그림과 비유로 — 모델링에 쓸 수 있을 만큼 이해하는 것이다.

1. 출발점: 기존 memory 가 막힌 곳

Transformer inference 를 아는 독자에게 익숙한 그림부터 시작하자. attention 은 과거 토큰의 key/value 를 **KV cache** 에 그냥 쌓아두고 (append), 새 query 가 오면 전부 훑어 비슷한 것을 찾는다 (lookup). 정확하지만 문맥이 길어질수록 cache 가 선형으로 커진다 — 100 만 토큰이면 layer 마다 GB 단위다.

반대편에는 **linear recurrent model**(Mamba, DeltaNet 계열) 이 있다. 이들은 과거를 고정 크기 state 하나에 압축한다. 크기가 안 커지니 싸지만, 긴 역사를 작은 행렬 하나에 옥여넣으니 언젠가 내용이 뭉개진다.

여기서 **TTT**(Test-Time Training, 이 계열의 직전 논문) 가 던진발상이 Titans 의 뿌리다.

직관 memory 를 “값을 적어두는 표” 가 아니라 “작은 모델” 이라고 보자. 그 작은 모델의 **weights** 가 곧 **state** 다. 새 토큰이 들어오면 그 모델을 그 토큰으로 **한 걸음 학습**시킨다 (gradient 한 번). 나중에 뭔가 물어보면 (query) 그 모델에 통과시켜 (forward) 답을 읽는다. 즉 “기억한다 = 작은 모델을 조금 더 학습시킨다”, “떠올린다 = 그 모델에 물어본다”.

그런데 TTT 에는 세 가지 결핍이 있었다. Titans 는 이 셋을 정확히 겨냥한다.

- **지우는 법이 없다.** TTT 의 내부 학습은 순수 SGD 라서 계속 쓰기만한다. state 는 고정 크기인데 무한히 덧쓰니 결국 포화한다. eviction(퇴거) 이 없다.
- **optimizer 가 기억을 못 한다.** 매 토큰의 갱신이 그 토큰 하나의 gradient 만 본다. “방금 놀라운 사건이 있었으니 그 직후 토큰들도 같이중요하다” 는 시간적 흐름이 갱신 규칙에 없다.
- **deep memory 가 검증 안 됨.** memory 를 깊은 MLP 로 둘 수 있다고만했지, 깊이가 실제로 뭘 사주는지 실험이 없었다.

비유 TTT 의 memory 는 “받아쓰기만 하고 지우개는 없는 학생, 그리고 방금 뭐가 놀라웠는지는 다음 문장에서 까먹는 학생” 이다. Titans 는 이 학생에게 (1) 지우개 (forgetting), (2) 놀란 여운을 이어가는 집중력 (momentum), (3) 더깊은 노트 (deep MLP) 를 쥐여준다.

한편 DeltaNet 계열은 정반대 선택을 했다 — memory 를 행렬 하나로 묶어 두는대가로 정확한 병렬 학습 공식을 얻었지만, memory 는 얇은 선형이었다. 그래서 Titans 이전의 지

형에는 가운데가 비어 있었다: 한쪽엔 “지울 줄 아는 얇은 선형 memory” (Gated DeltaNet), 다른 쪽엔 “깊지만 못 지우고 여운도없는 memory” (TTT). Titans 는 그 가운데를 채운다.

2. 큰 그림: 모든 sequence model = 구조 + write + read

Titans 는 세상의 모든 sequence model 을 세 부품으로 분해해서 본다.

- **memory 구조** — 무엇으로 과거를 담는가 (KV 표? 행렬 하나? 작은 MLP?)
- **write(쓰기)** — 새 토큰을 어떻게 반영하는가
- **read(읽기)** — 저장된 것에서 어떻게 답을 꺼내는가

이 렌즈로 보면 익숙한 것들이 정리된다. Transformer 는 “압축 없이 KV 를 append 하는 write + 유사도 검색 read” 다. linear attention 은 “ vk^T 을 행렬 하나에 더하는 write” 라 넘칠 게 예정돼 있다. Titans 는 여기에 **깊은 MLP memory + 학습된 optimizer** 로써의 **write** 를 넣는다.

또 하나의 핵심 철학이 이 계열 전체의 성격을 정한다.

직관 훈련 데이터를 통째로 외우는 건 나쁘다 (일반화 · 프라이버시 손해). 그래서 Titans 는 pre-training 에서 “**외우는 방법**”만 배우고 (meta-learning), 실제 내용 암기는 **test time** 에 실제 문맥위에서만 일어나게 한다. 제목 “Learning to Memorize at Test Time” 이 바로 이 두 층 구조 선언이다.

이 두 층을 이 권 내내 이렇게 부른다.

기호 풀이 **outer loop**(느린 층) = pre-training. 여기서 학습되는 것을 통칭 Θ (slow weights) 라 하고, inference 중에는 **동결**된다. **inner loop**(빠른 층) = test time. 여기서 실제로 움직이는 것이 memory 의 fast weights W_t 다. t = 토큰 위치 (시간 첨자). “outer 에서 배우는 방법을 익히고, inner 에서 그 방법으로 지금 문맥을 외운다.”

3. Titans 의 write: surprise + momentum + forgetting

write 는 세 조각을 차곡차곡 쌓아 만든다. 하나씩 직관부터 본다.

3.1 memory 가 푸는 문제: key 를 넣으면 value 가 나오게

먼저 memory 에게 시킬 일을 정한다. 토큰 x_t 를 두 가지로 변환한다 — 검색용 열쇠 k_t (key) 와 그때 꺼낼 내용 v_t (value). 그리고 memory (작은 MLP) 가 “ k_t 를 넣으면 v_t 가 나오도록” 스스로를 맞춰가게 한다.

$$k_t = W_K x_t, \quad v_t = W_V x_t, \quad \ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2.$$

기호 풀이 $x_t = t$ 번째 입력 토큰 벡터 (폭 d). W_K, W_V = key/value 를 만드는 projection 행렬 ($d \times d$), **outer** 에서 배우고 **inference** 엔 고정. k_t = 검색 열쇠, v_t = 그 열쇠에 붙일 내용. $\mathcal{M}(\cdot; W)$ = memory 신경망 (작은 MLP), W = 그 안의 fast weights (진짜 state). ℓ = inner loss, “memory 가 k_t 로 만든 답과 정답 v_t 가 얼마나 다른가”의 제곱 오차.

직관 attention 이 하는 “비슷한 key 엔 연관된 value 를 돌려줘라” 를, 표에 적어두는 대신 작은 모델의 **weights** 안에 새겨 넣겠다는 것이다. ℓ 이 작다 = memory 가 이 key-value 짝을 이미 잘 안다.

3.2 surprise = 얼마나 틀렸나

이제 “이 토큰을 얼마나 세게 외울까?” 를 정해야 한다. 답: **memory** 가 이 토큰에 얼마나 놀랐는가.

직관 memory 가 이미 아는 내용이면 놀랄 게 없다 (작게 반영). 처음 보는 · 예상 밖 내용이면 크게 놀란다 (세게 반영). “놀란 정도”의 수학적 이름이 바로 **gradient**의 크기다 — loss 가 가파르게 틀린 방향일수록 gradient 가 크다. 이것을 **momentary surprise** (순간 놀람) 라 부른다.

가장 기본형 write 는 gradient 한 걸음이다.

$$W_t = W_{t-1} - \eta_t g_t, \quad g_t = \nabla_W \ell(W_{t-1}; k_t, v_t).$$

기호 풀이 W_t = 토큰 t 까지 반영한 memory weights. W_{t-1} = 직전 상태. g_t = momentary surprise, 즉 지금 loss 의 gradient (놀란 방향과 크기). η_t = **inner learning rate**, “이번엔 얼마나 세게 배울까”를 정하는 토큰별 게이트 (뒤에서 outer 가 만드는 함수가 산출).

평이하게 읽으면: “직전 memory 에서 지금 토큰이 틀린 만큼 (gradient) 을, 학습 강도 η_t 로 반영해 새 memory 를 만든다.” 익숙한 어휘로는 **read-modify-write** 다 — append 가 아니라, 현재 저장값을 먼저 읽어오차만큼만 고쳐 쓴다.

시스템 모델링 관점 memory 가 선형 (W 가 $d \times d$ 행렬) 이면 이 식은 정확히 rank-1 outer-product write ($W_{t-1}k_t - v_t$) $k_t^\top$ 로 퇴화한다 — DeltaNet/TTT 가 이미 서 있던 자리다. 즉 이 기본형은 “KV append 한 줄”이 “rank-1 GEMM write 한 번”으로 바뀐 것. 데이터 의존성은 $W_{t-1} \rightarrow W_t$ recurrence 에 있다 (순차).

3.3 momentum = 최근 놀란 방향의 관성

순간 놀람만으로는 실패하는 순간이 있다.

직관 큰 사건에 한 번 놀라면, 바로 그 다음 토큰에서는 이미 loss 가 낮아져 gradient 가 작다. 그래서 놀라운 사건 직후에 이어지는 중요한 토큰들이 약하게 저장된다. 실제 기억은 안 그렇다 — 놀라운 순간은 한동안 주의를 붙들어 그 구간 전체를 함께 기억하게 만든다. 이 “여운” 을 넣는 장치가 **momentum** 이다.

$$S_t = \beta_t S_{t-1} - \eta_t g_t, \quad W_t = W_{t-1} + S_t.$$

기호 풀이 S_t = **past surprise**, 최근 놀람들을 누적한 버퍼 (W 와 같은 크기 · 모양). β_t = **surprise decay**(0~1), 과거 여운을 얼마나 이어갈지의 토큰별 게이트. $\beta_t \rightarrow 1$ 이면 여운을 온전히 이어가고 (같은 문맥이 계속), $\beta_t \rightarrow 0$ 이면 여운을 끊는다 (문맥이 바뀔). 나머지 기호는 앞과 동일.

평이하게 읽으면: “이번에 write 할 양 S_t 는 = 직전까지의 놀란관성 ($\beta_t S_{t-1}$) + 지금 새로 놀란 양 ($-\eta_t g_t$). 그걸 memory 에 더한다.” 이것은 문자 그대로 **SGD-with-momentum** 의 **momentum buffer** 다. Titans 의 첫 번째 발명은 “optimizer 가 원래 갖고 있던 momentum 이라는 내부 상태를, 아예 sequence model 의 recurrent state 로 승격시킨” 것이다.

비유 S_t 는 “관심의 관성” 이다. 방금 뭔가에 크게 놀랐으면 (S 가 큼) 그 방향으로 잠시 계속 밀려간다. β_t 는 그 관성이 얼마나 오래가는지를 문맥에 맞춰 조절하는 다이얼.

3.4 forgetting = 조금씩 지우기 (weight decay)

수백만 토큰을 다루면 깊은 memory 라도 언젠가 가득 찬다. 마지막 조각은 **지우개**다. 그 정체는 놀랍도록 단순하다 — 매 토큰 memory 를 조금씩 **곱해서 줄이는** weight decay.

이 셋을 다 합친 것이 이 계열의 기준 갱신식 (이 책에서 **master update** 라 부른다) 이다.

$$S_t = \beta_t S_{t-1} - \eta_t g_t, \quad W_t = \alpha_t W_{t-1} + S_t.$$

기호 풀이 α_t = **남기는 비율** (retention, 0~1) 인 토큰별 게이트. $\alpha_t \rightarrow 1$ 이면 과거 memory 를 거의 다 보존한 채 덧쓰고, $\alpha_t \rightarrow 0$ 이면 memory 를 거의 통째로 비운다. (주의: Titans 원 논문의 α 는 반대로 “잊는 비율” 이라 방향이 뒤집혀 있다 — 이 책은 “남기는 비율” 로 통일한다.) $S_t, \beta_t, \eta_t, g_t$ 는 앞과 동일.

평이하게 읽으면: “새 memory = 과거를 α_t 만큼 남긴 것 + 이번에 놀란 양 S_t .” 딱 세 개의 다이얼 (η_t 학습 강도, β_t 여운, α_t 남김) 로 write 전체가 굴러간다.

핵심 Titans 의 write = “**gradient 한 걸음 + momentum + weight decay**” 라는 **optimizer 한 스텝**이다. 그리고 그 optimizer 의 하이퍼파라미터 세 개가 전부 토큰마다 학습된 게이트로 산출된다. 2 권 (G02) 에서 optimizer 를 (상태, 갱신식, 비용) 을 가진 객체로 봤다면, Titans 는 그 객체를 통째로 모델 안에 집어넣고 다이얼 세 개를 “배운 함수” 로 바꾼 것이다.

시스템 모델링 관점 이 layer의 **inner state**는 두 덩어리다 — memory weights W_t 와 momentum buffer S_t . 둘 다 크기가 같다. memory가 폭 d 짜리 L_M 층 MLP면 파라미터 수 $P_M \approx L_M d^2$ 개. 따라서 state 총량 = $2P_M$ 개의 숫자 — **momentum 이 state를 정확히 2배로 만든다**. 데이터 의존성: W_t 와 S_t 모두 직전 값에 의존하는 순차 recurrence. 토큰당 write 연산: key/value projection 2번 + memory forward 1번 + gradient(backward) 1번 + S, W 의 원소별 갱신. 뒤 3.7에서 정량화한다.

세 성분의 실제 기여 순위 (원문 ablation)는 **weight decay** > **momentum** > **convolution** > **persistent memory**다. 지우개가 가장 중요하다는 것이 결론이다.

4. read: 그냥 통과시키기

읽기는 훨씬 단순하다. weight를 바꾸지 않는 순수 forward pass다.

$$q_t = W_Q x_t, \quad y_t = \mathcal{M}(q_t; W_t).$$

기호 풀이 q_t = query (질문 벡터). W_Q = query projection ($d \times d$), 역시 outer에서 배우고 고정. $\mathcal{M}(q_t; W_t)$ = 현재 memory에 q_t 를 통과시킨 출력. y_t = 이 위치의 read 결과.

평이하게 읽으면: “질문 q_t 를 지금 memory에 물어보면 답 y_t 가 나온다.” read가 state를 안 바꾼다는 점은 KV cache lookup과 똑같다 — 다른건 lookup이 표 검색이 아니라 작은 MLP forward라는 것뿐.

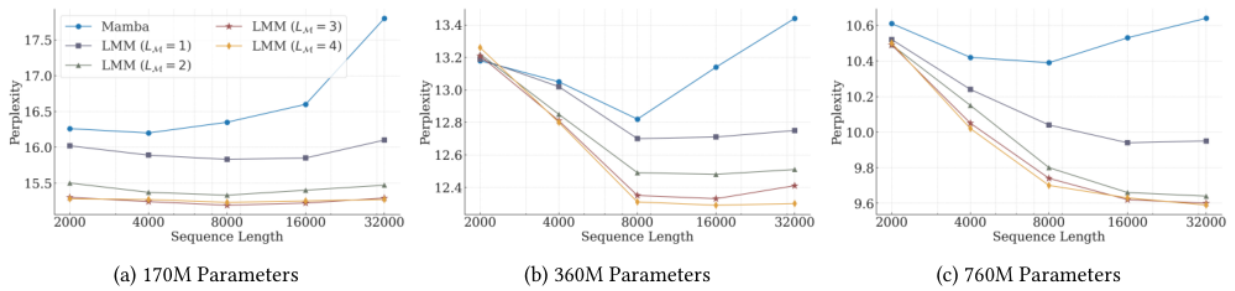
시스템 모델링 관점 write와 read가 명시적으로 분리된 두 연산인 것이 이 계열의 핵심 구조다. write는 순차 (state 갱신), read는 그 시점 state에 대한 stateless forward. decode한 스텝은 이제 read만이 아니라 write(forward+backward+갱신) + read를 모두 포함한다 — 6절과 7절에서 이 뒤집힘을 다룬다.

5. 왜 깊은 MLP memory인가

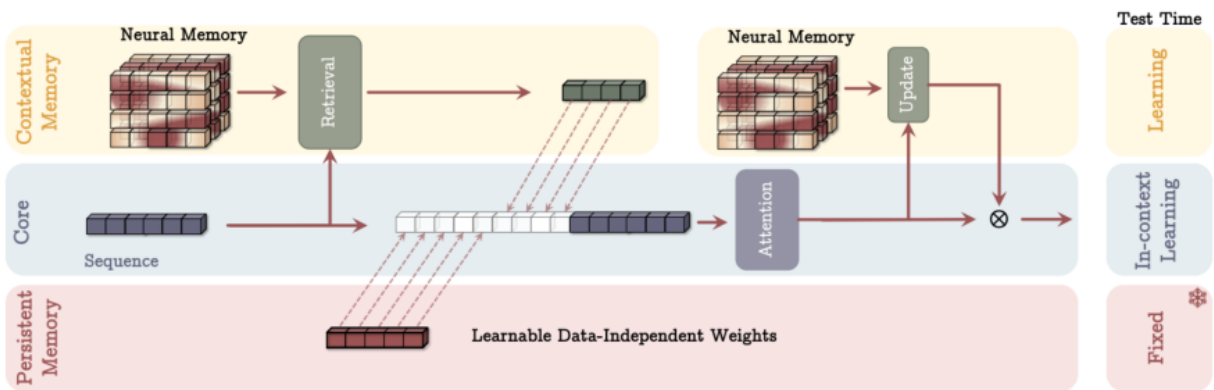
memory를 그냥 행렬 하나 ($W \in \mathbb{R}^{d \times d}$)로 두면, “key $\rightarrow$ value” 관계가 **선형 사상**으로 압축 가능하다”고 가정하는 셈이다. 현실의 긴 문맥은 그렇게 단순하지 않다.

직관 얇은 선형 memory = 직선 자 하나로 세상을 근사하는 것. 깊은 MLP memory = 곡선까지 담을 수 있는 노트. key-value 구조가 비선형이어도 저장할 수 있다는 게 deep memory의 논거다. 실측으로도 memory를 선형으로 바꾸면 long-context 점수가 92.68에서 85.34로 급락한다.

아래 그림이 그 실증이다. memory 깊이 L_M 을 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ 로 키우면, 모든 sequence 길이에서 perplexity (낮을수록 좋음)가 개선되고, 특히 길이가 늘 때의 열화가 완만해진다.



memory 깊이 L_M 이 perplexity 에 주는효과 (170M/360M/760M). 깊을수록 모든 길이에서 낮은 perplexity, 특히 긴길이에서 강건. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.7 —원저자 그림, 본서 결과 아님.



Memory as Context (MAC) 구조. 세 branch — core(attention), contextual(long-term) memory, persistent memory. 검색해 온 과거를 현재 segment 앞에 붙여 attention 에 넣고, attention 출력을 memory 에 write. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.2 —원저자 그림, 본서 결과 아님.

기호 풀이 L_M = memory MLP 의 층 수 (깊이). $L_M = 1$ 이면 선형 memory, ≥ 2 면 deep memory. perplexity = 언어모델이 다음 토큰을 얼마나 잘 맞히는지의 척도 (낮을수록 좋음).

한계 깊을수록 좋지만 공짜가 아니다 — memory 가 깊어지면 훈련 throughput 이선형으로 떨어진다. 그리고 원 논문 v1 은 memory MLP 의 폭·activation 같은세부를 명시하지 않았다 (재구현자가 채워야 하는 공백). 이 계열 후속작들은결국 2층 residual MLP 를 표준 deep memory 로 굳힌다.

6. memory 와 attention 을 합치는 3 가지: MAC / MAG / MAL

attention 은 정밀한 단기 기억 (방금 본 것을 또렷이), Titans 의 memory(LMM, long-term memory module) 는 서서히 잊는 장기기억이다. 이 둘을 어떻게 배치하느냐로 세가지 설계가 나온다.

MAC (Memory as Context). 입력을 크기 C 의 조각 (segment) 으로자른다. segment가 오면 → 직전까지 갱신된 memory에서 관련 과거를읽어 (read) → 그걸 현재 segment 앞에 붙여 attention 을 돌리고 →그 **attention 출력만 memory 에 쓴다 (write)**. 위 그림의 “read → attention → write” 흐름이 이것이다.

직관 attention 이 “지금 이 순간 장기 기억이 필요한가?” 를 토큰 단위로판단한다. 그리고 attention 이 거른 표현만 memory 에 쓰이므로, attention 이일종의 **write filter** 가 되어 쓸모없는 토큰으로 memory 가 넘치는 걸막는다.

MAG (Memory as Gate). 자르지 않는다. 두 갈래를 나란히 돌린다 — 하나는 sliding window attention(SWA, 최근 window 만 보는 attention), 하나는 전체 흐름을 도는 memory. 두 출력을 학습된 게이트로 섞는다. SWA 가또렷한 단기, memory 가 흐릿한 장기.

MAL (Memory as Layer). memory 를 그냥 한 layer 로 attention 앞에직렬로 쌓는다. 사실 기존 하이브리드 모델 (Samba, Griffin 등) 이 다 이모양이다. 그런데 논문은 **이게 가장 약하다고** 못박는다 —파이프라인의 힘이 각 단의 힘으로 상한되어 attention 과 memory 의 상보성을못 살린다는 것.

세 변형 모두 **persistent memory** 를 포함한다.

기호 풀이 persistent memory P = 입력과 무관하게 **학습되는 고정 prefix 토큰** N_p 개 ($N_p \times d$ 크기). 문맥이 뭐든 항상 sequence 맨 앞에붙는다. “task 를 푸는 방법에 대한 지식”(입력마다 바뀌면 안 되는 것) 을담고, test time 엔 동결된다. C = MAC 의 segment 크기.

핵심 실험 결론은 **MAC $\approx$ MAG > MAL**. 모듈은 같고 배치만다르니, 이 격차는 순수 하게 합성 설계의 몫이다. 기존 하이브리드가 전부 MAL 모양이라는 점에서, 이 결과는 하이브리드 설계 관행 자체에 대한반증이다.

이 배치가 실제로 얼마나 강한지는 초장문 벤치마크에서 드러난다. 아래 BABILong 에서 작은 MAC 모델이 훨씬 큰 GPT-4·Llama3.1-70B 등을 앞서고, 200 만 토큰 너머까지 정확도를 유지한다. 이 계열의 “>2M context” 헤드라인이 전부 여기서 나온다.

7. 누가 무엇을 배우나: outer vs inner 정리

이 계열에서 가장 헷갈리는 지점을 표로 못박는다. “그 게이트는누가 학습하는가?” 에 부품 단위로 답한다.

구성 요소	소속	언제 갱신	크기 감각
projection W_K, W_V, W_Q	outer(느림)	pre-training 에서만	inner loss 의 하이퍼 파라미터
게이트 산출 head ($\eta_t, \beta_t, \alpha_t$ 를 만드는 함수)	outer	pre-training 에서만	inference 때 토큰→ 게이트 3 개 산출

구성 요소	소속	언제 갱신	크기 감각
persistent tokens P	outer	pre-training 에서만, test 때 동결	$N_p \times d$
attention·conv·normalization·LM head	outer	pre-training 에서만	통상 backbone
memory weights W_t	inner(빠름)	매토큰, inference 중에도	$P_M \approx L_M d^2$
momentum buffer S_t	inner	매 토큰	W_t 와 같은 크기 — state 2 배의 주범

직관 게이트 세 개 $(\eta_t, \beta_t, \alpha_t)$ 의 값은 inference 중매 토큰 새로 계산된다. 하지만 그 값을 만드는 함수 (head 의 weights) 는 pre-training 에서만 배우고 이후 고정이다. 즉 “얼마나 세계배울지 정하는 방법” 은 미리 배우고, “이번 토큰엔 실제로 얼마나 세계” 는 그때그때 정한다.

비유 outer = 운전 교습소에서 “어떤 상황에 얼마나 밟을지” 를 배우는 것 (느림, 한 번). inner = 실제 도로에서 매 순간 페달을 밟는 것 (빠름, 매 토큰). 배운 것은 “정책”, 바뀌는 것은 “그 순간의 페달 세기”.

시스템 모델링 관점 outer gradient 는 펼쳐진 (unrolled) inner 갱신들을 관통해서 흐른다 — 매 토큰의 write 가 forward graph 의 일부이고, autodiff 가 그전체를 backprop 한다. 즉 “ W_K 를 조금 바꾸면 $\rightarrow$ 매 토큰 gradient 가 바뀌고 $\rightarrow$ memory 궤적 전체가 바뀌고 $\rightarrow$ 최종 loss 가 바뀐다” 의 연쇄로 W_K 가 학습된다. 이것이 “optimizer 를 미분한다” 의 실체. **이권에서 독자의 세계가 뒤집히는 지점**: decode = “read-only forward + KV append” 라는 불변식이 폐기된다. decode 한 스텝이 이제 forward + backward + optimizer step + read 다.

8. 훈련은 어떻게 병렬화되나 (개념만)

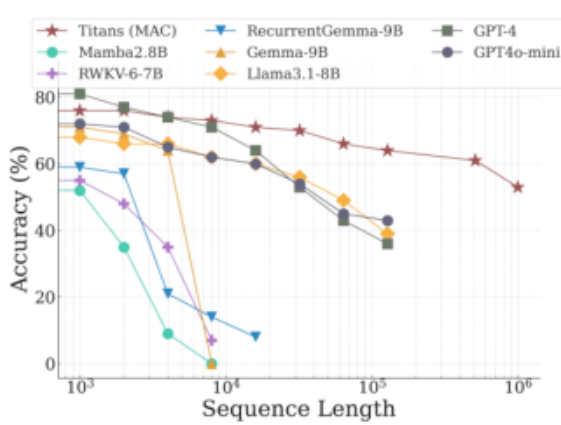
앞의 갱신식은 토큰을 하나씩 순차로 처리한다 — GPU 에서 최악의 형태 (matmul 없음). Titans 는 어떻게 이걸 병렬 GEMM 위에 올릴까? 증명 없이 그림으로만 본다.

핵심 트릭은 **chunkwise** 근사다.

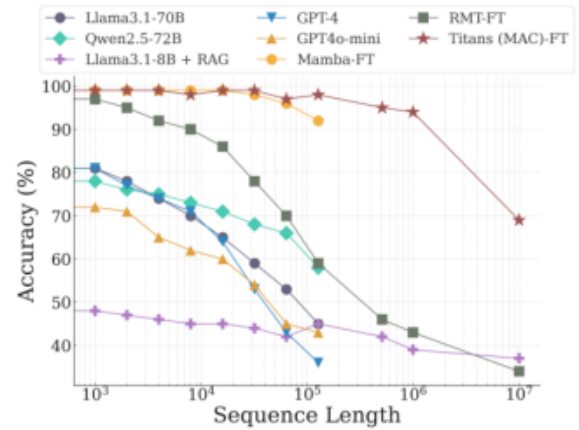
직관 sequence 를 크기 C 의 chunk 로 자른다. chunk 안에서는 “각 토큰의 gradient 를 전부 **chunk 시작 시점의 같은 memory 상태에서 평가**” 한다. 그러면 chunk 안 C 개를 동시에 (matmul 로) 계산할 수 있다. chunk 와 chunk 사이만 순차로 넘긴다. 순차 임계 경로가 L (토큰 수) 에서 L/C (chunk 수) 로 줄어든다.

기호 풀이 C = chunk(mini-batch) 크기. L = 전체 sequence 길이. chunk 시작 상태에서 gradient 를 평가하는 것을 “stale(살짝 낡은) snapshot 근사” 라 부른다 — 정

Model	S-NIAH-PK				S-NIAH-N				S-NIAH-W			
	2K	4K	8K	16K	2K	4K	8K	16K	2K	4K	8K	16K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4	0.0
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6	0.0
Titans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8	80.6
Titans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6	95.2
Titans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2	88.2
Titans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0	90.4

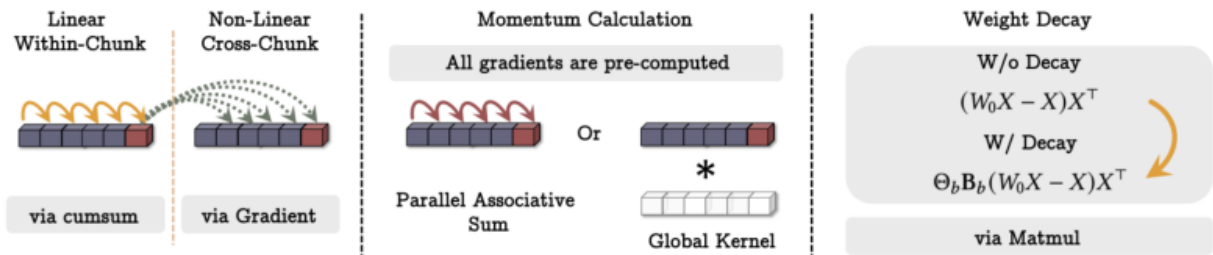


(a) Few-shot Setup



(b) Fine-Tuning Setup

BABILong 정확도 대 sequence 길이. (위) few-shot 에서 Titans(MAC) 가 GPT-4 등 훨씬 큰 모델을 앞섬. (아래) fine-tuning 에서 작은 MAC 가 100 만 토큰 너머까지 정확도 유지. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.6 — 원저자 그림, 본서 결과 아님.



inner-loop 갱신을 가속기 친화 연산으로 바꾸는 세 조각. (왼쪽) chunk 안 갱신은 선형이라 cumsum 으로, chunk 경계는 비선형이라 gradient 로. (가운데) momentum 은 gradient 를 미리 계산하면 1 차 선형 recurrence 가 되어 scan 으로. (오른쪽) weight decay 는 누적 곱을 대각 행렬로 접어 matmul 에 흡수. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.1 — 원저자 그림, 본서 결과 아님.

확한 순차 계산과 살짝 다르지만 훨씬 빠르다.

결과 구조가 Titans 의 표현력-병렬성 협상이다.

핵심 chunk 안은 선형 (matmul + scan 으로 병렬), **chunk 경계는비선형** (경계에서 갱신된 MLP weights 가 다음 chunk 에 비선형으로 재진입). DeltaNet 계열은 정확한 병렬 공식을 위해 전부 선형으로 남았지만, Titans 는 chunk 경계에 일부러 비선형을 남겨 표현력을 얻는다.

시스템 모델링 관점 momentum 갱신 ($S_t = \beta_t S_{t-1} - \eta_t g_t$) 은 gradient 들을미리 계산해 두면 정확히 **Mamba 의 selective scan 과 같은 1 차 선형 recurrence** 다 — 따라서 익숙한 parallel scan kernel 로 chunk 안 모든 S_t 를 log-depth 에 얻는다. weight decay 의 누적 곱은 대각 행렬로 접혀 matmul 에 흡수된다. **한 가지 함정**: 훈련은 $C > 1$ 의 chunk 근사를쓰고 decode 는 $C = 1$ 의 정확한 순차 write 를 쓴다. C 는 계산되는 함수자체를 바꾸는 의미적 하이퍼파라미터라서, $C_{\text{train}} \neq C_{\text{decode}}$ 이면 두 궤적이 일반적으로다르다 — 이 train/serve 불일치가 이 계열의 미해결 문제 중 하나다.

9. 시스템 관점: state 가 커지고, decode 가바뀐다

이 권에서 시스템 엔지니어가 가장 챙길 절이다.

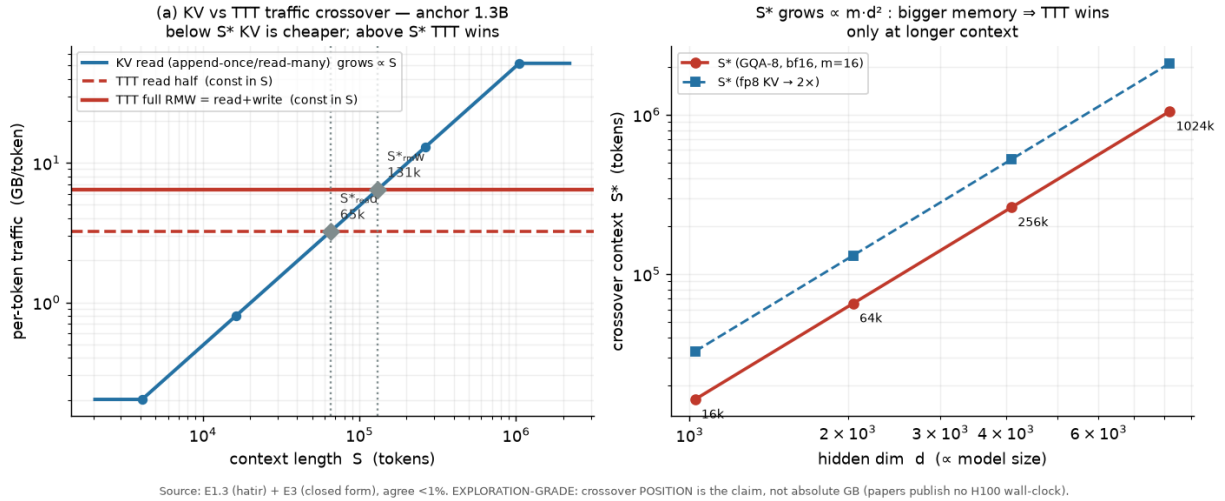
9.1 state 수학: KV cache 와의교환

항목	크기 식	$d=1024$, 2 층 memory, $L=2\text{M}$ 토큰
LMM state ($W_t + S_t$)	$2P_M \approx 2L_M d^2$	약 4M 값 $\approx 8\text{MB}$ — 길이 불변
attention KV cache	$2Ld$	약 4G 값 $\approx 8\text{GB}$ — 길이 비례

직관 이 표 한 줄이 Titans 의 존재 이유다. attention 의 KV cache 는 문맥이 길수록 선형으로 커져 2M 토큰이면 layer 당 GB 급이 된다. Titans memory 의 state 는문맥 길이와 **무관하게 일정**하다 (위 예에서 약 8MB 고정). BABILong 의 >2M 능력이 정확히 이 교환 위 에 서 있다.

주의 공짜가 아니다. momentum buffer S_t 가 memory state 를 정확히 2 배로만든다. 같은 폭의 Gated DeltaNet(단일 $d \times d$ state) 대비로는 약 $2L_M$ 배 (2 층이면 약 4 배). 그리고 그 buffer 는 decode 스텝사이에 반드시 이월돼야 한다.

아래는 이 책의 자체 실험이다. 토큰당 state 트래픽을 KV(문맥에 비례해증가) 와 TTT 류 fast-weight(문맥 무관 상수) 로 비교하면, 어떤 문맥 길이 S^* 에서 두 곡선이 교차한다 — 그보다 짧으면 KV 가 싸고, 길면 fast-weight 가 이긴다.



KV vs TTT 토큰당 트래픽 교차점 S^* (anchor 1.3B). S^* 보다 짧으면 KV cache 가 싸고, 길면 fast-weight state 가 이긴다. 오른쪽은 모델 크기에 따른 S^* 스케일링. 출처: 본서 자체실험 (neural-memory-study, E1.3+E3) — 교차점의 위치가 주장 (exploration-grade).

기호 풀이 P_M = memory 파라미터 수 $\approx L_M d^2$. d = 폭, L = 문맥 길이, L_M = memory 길이. S^* = KV 와 fast-weight 의 토큰당 트래픽이 같아지는 교차 문맥 길이.

9.2 decode 의재정의

시스템 모델링 관점 decode 한 스텝은 이제 read-only lookup 이 아니라 $2P_M$ 전체에 대한 read-modify-write 다 — W_t 와 S_t 를 읽고, 원소별로 갱신해, 도로 쓴다. 토큰당 연산량세기 (단위 MAC): write $\approx$ key/value projection $2d^2$ + memory forward $\approx P_M$ + gradient(backward) $\approx 2P_M + S$, W 원소별 갱신 $\approx 2P_M$; read $\approx$ projection + forward $\approx d^2 + P_M$. 합계 토큰당 대략 $4-5 \times P_M$ MAC(FLOP 으로는 약 $8-10 \times P_M$) — **문맥 길이와 무관한상수**. 토큰당 state 트래픽도 $O(P_M)$ 이므로 이 layer 의 arithmetic intensity 는 낮은 상수에 고정된다 (memory-bound). KV cache 스트리밍이 있던 자리에 weight-state RMW 스트리밍이 들어선 셈인데, 이제그 트래픽이 문맥 길이와 무관하다는 점이 다르다.

9.3 serving 의 새질문들

주의 shared-weight 전제가 깨진다. request 마다 W_t, S_t 가 다르므로, naive multi-tenant batching 은 request 별 weight 사본을 요구한다 — batched decode 가 shared-weight GEMM 이 아니라 per-sample weights 의 grouped GEMM/bmm workload 가 된다. 그 밖에도: retention 게이트 α_t 는 **학습된 eviction policy**(무엇이 언제 지워지는지 runtime 에 불투명), state 가 optimizer 궤적의 산물이라 accumulation 순서 · 정밀도 numerics 가 “cache” 를 표류시킬 수 있음 (KV 엔 없던 결정성 문제), speculative decoding 의 rollback 은 (W_t, S_t) 궤적의 snapshot/복원이라는 비자명한 연산, 그리고 문맥이 weights 에 흡수되므로 prompt privacy · multi-tenant 격리 질문까지. 원 논문은 이 중 무엇도 다루지않는다.

한계 이 논문을 포함해 이 계열 전체에 **decode wall-clock** 수치가 없다. 위 serving 논의 구조에서 연역한 것이지 측정이 아니다. 발표된 효율수치는 전부 **훈련** throughput 이고, 그마저 “LMM 이 Mamba-2/Gated DeltaNet 보다 약간 느리다” 까지다 (fused kernel 부재가 원인으로 지목).

10. 한 발 물러서서: Titans 는 기존 계보 전부를 품는다

Titans 의 갱신식은 다이얼 몇 개를 끄면 기존 모델들이 된다.

직관 momentum 을 끄면 ($\beta_t = 0$) → **Gated DeltaNet**. forgetting 과 momentum 을 둘 다 끄면 → **TTT**. 같은 loss·형식을 쓴다는 이유로 **RWKV-7**, **Longhorn** 도 특수 사례로 회수된다. 즉 이 계보전체가 하나의 설계 공간 위의 점들이고, Titans 는 그 공간에서 (momentum, deep memory, 비선형 chunk 경계, retention gate) 네 축을 모두 켜놓은 점이다.

한계 바로 그래서 열리는 질문 — “왜 하필 l_2 loss 인가? 왜 하필 momentum+decay 인가? 그 축들 위의 다른 점은?” Titans 는 “기존 모델전부가 내 특수 사례”라는 통일은 보여주지만, 그 설계 공간의 **좌표축이 무엇인지는 이름 붙이지 않았다**. 그 이름 붙이기가 다음권의 출발점이다.

요약 Titans 의 memory 는 “읽으며 계속 학습시키는 작은 MLP” 다. write = **surprise**(얼마나 틀렸나) + **momentum**(놀란 여운) + **weight decay**(조금씩 지우기) 라는 학습된 optimizer 한 스텝이고, 그 다이얼 세개 ($\eta_t, \beta_t, \alpha_t$) 는 outer 가 만든 함수가 토크마다 산출한다. read 는 갱신 없는 forward. memory 를 attention 과 합치는 법은 **MAC** $\approx$ **MAG** > **MAL** 세 가지. 시스템 관점에서 state 는 **길이불변**이라 초장문에 강하지만, momentum 이 state 를 2 배로 키우고 **decode** 가 **read-modify-write(backward 가 decode 안으로)** 로 재정의되며, shared-weight batching 이 깨진다. 훈련은 chunk 안 선형·chunk 경계 비선형으로 병렬화된다.

다음 권 예고 (G04 · Miras). Titans 가 “기존 모델 전부가 내 특수사례” 라고 말한 그 설계 공간의 **좌표축에 이름을 붙이고** (objective / retention / 아키텍처 / 알고리즘), 왜 하필 l_2 였는지를 다시 묻는것이 다음 권 Miras 다.